The Evolution of a C– Compiler

Lexical Analyzer (A1) and Recursive-Descent Parser (A2)

Abdullah Shoaib
23L-0569

5 October 2026

Abstract

This report follows the construction of a compiler front end for C–, a small subset of C++. Part I covers the lexical analyzer: the token classes, the regular definitions of the complex tokens, a transition diagram that recognizes every token of the language, and the notes that go with the implementation. Part II covers the parser: a context-free grammar for C–, its LL(1) form, and a recursive-descent implementation that reuses the lexer. The closing section traces how the source tree grew from the lexer to the parser.

Contents

1 Introduction **1**

2 Lexical Analysis **1**
2.1 Token classes 1
2.2 Regular definitions 2
2.3 Transition diagram 2
2.4 Implementation notes 2

3 Syntax Analysis **4**
3.1 A context-free grammar for C– 4
3.2 Making the grammar LL(1) 5
3.3 Recursive descent 5

4 How the codebase evolved **6**

1 Introduction

C– is a deliberately small language. It supports only two data types, `int` and `char`; there are no arrays and no strings. The constructs it does support are

- declarations (`int x;`, `char c = 'a';`),
- expressions and store statements,
- selection with `if`–`else`,
- iteration with `while`,
- functions (definitions, parameters and calls).

Relational operators are part of the language; logical operators (`&&`, `||`, `!` as a unary operator) are not. Two special keywords, `in` and `out`, are used for input and output. The work is split into two stages:

Lexer the lexical analyzer (this part),

Parser a recursive-descent parser that reuses the lexer.

2 Lexical Analysis

2.1 Token classes

Every lexeme produced by the scanner belongs to one of the classes below.

Class	Token type	Examples
Identifier	`ID`	`x`, `total`, `factorial`
Keyword	`KEYWORD`	`int`, `if`, `while`, `in`, `out`
Integer literal	`INTEGER`	`0`, `42`, `1000`
Character literal	`CHAR`	`'a'`, `'Z'`, `'0'`
Arithmetic operator	`ARITHMETIC_OP`	`+`, `-`, `*`, `/`
Relational operator	`LT/LTE/GT/GTE/EQ/NEQ`	`<`, `<=`, `>`, `>=`, `==`, `!=`
`=` operator	`ASSIGNMENT_OP`	`=`
Delimiter	`DELIMITER`	`( ) { } ; ,`
End of file	`EOF`	—
Error	`ERROR`	an illegal character

2.2 Regular definitions

The tokens that have more than one lexeme are described by the following regular definitions, where `digit` and `letter` are the usual character classes.

Name	→	Regular definition
digit	→	[0-9]
letter	→	[A-Za-z]
under	→	_
id	→	letter (letter | digit | under)*
integer	→	digit digit*
charlit	→	' (any char except ' and newline) '
relop	→	< | <= | > | >= | == | !=
addop	→	+ | -
mulop	→	* | /

A lexeme matched by `id` is looked up in a reserved-word set; if it is found it becomes a `KEYWORD`, otherwise it stays an `ID`. The two-character relational operators (`<=`, `>=`, `==`, `!=`) are the reason the scanner needs one character of lookahead.

2.3 Transition diagram

Figure 1 is the transition diagram of the scanner. State q_0 is the start state; a state drawn with a double circle is an accepting state and is labelled with the token class it emits.

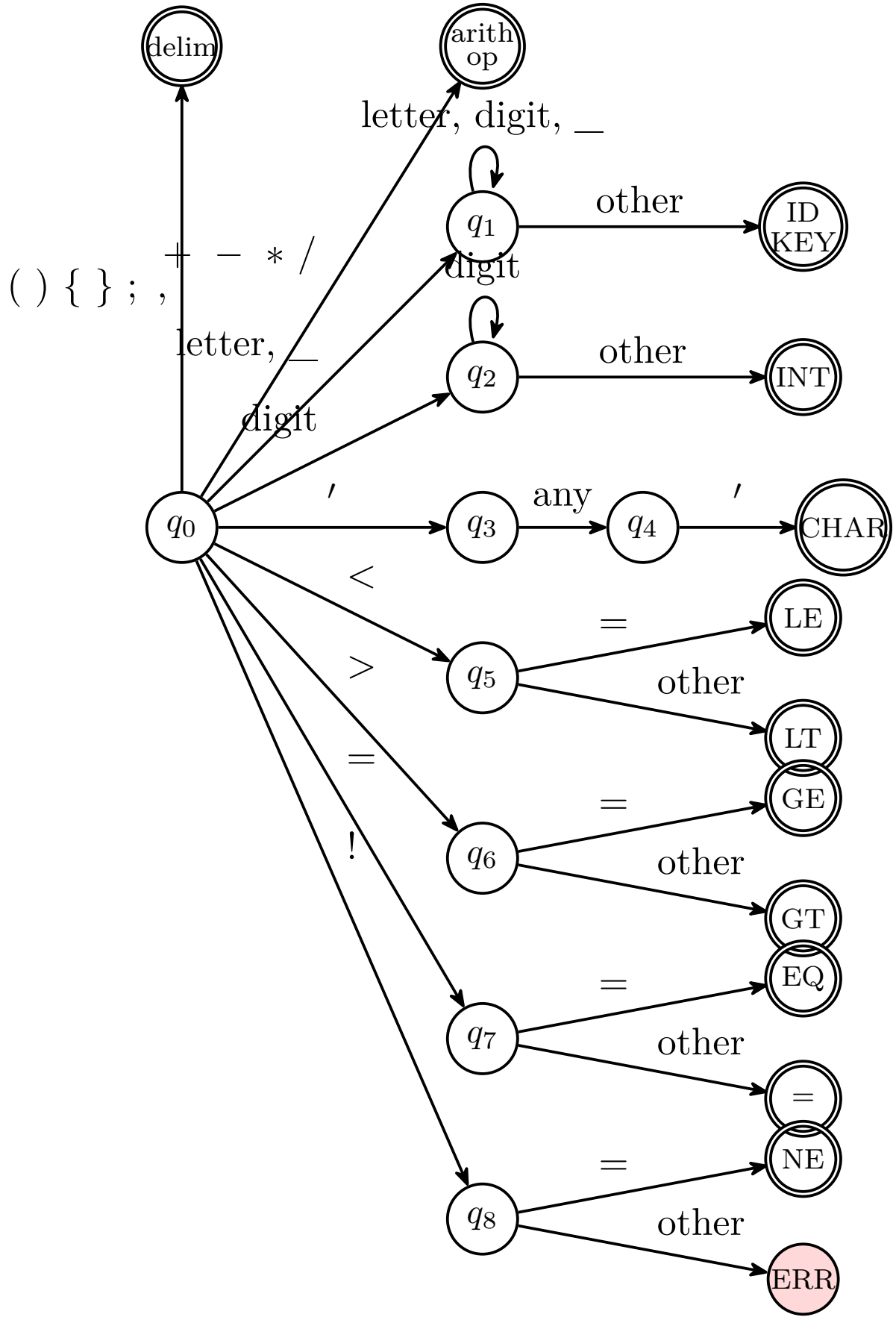


Figure 1: Transition diagram recognizing every token of C–. Whitespace is consumed between tokens and is not shown.

2.4 Implementation notes

The scanner is the `Lexer` class. It keeps the source text together with a position, a line and a column so that diagnostics can point at the offending location. Whitespace is skipped before each token, and the public interface is simply

```
bool hasMoreTokens();
Token nextToken();
```

A `Token` carries its type, its lexeme and the line/column where it starts. The interesting methods are the ones for the multi-lexeme tokens: `scanID` (which also classifies keywords), `scanInteger` and `scanChar`. The scanner reports two lexical errors directly: an unknown character and a stray `!` that is not followed by `=`.

3 Syntax Analysis

The parser reuses the lexer of Part I unchanged: it pulls one token at a time from the same `Lexer` object and decides which production to apply. Every decision can be made from the next single token, so a recursive-descent parser is a direct transcription of the grammar with one function per nonterminal.

3.1 A context-free grammar for C–

$$
\begin{aligned}
\mathit{Program} &\to \mathit{FuncList} \\
\mathit{FuncList} &\to \mathit{Func}\ \mathit{FuncList} \mid \varepsilon \\
\mathit{Func} &\to \mathit{Type}\ \texttt{ID}\ \texttt{(}\ \mathit{ParamList}\ \texttt{)}\ \mathit{Block} \\
\mathit{Type} &\to \texttt{int} \mid \texttt{char} \\
\mathit{ParamList} &\to \mathit{Param}\ \mathit{ParamTail} \mid \varepsilon \\
\mathit{ParamTail} &\to \texttt{,}\ \mathit{Param}\ \mathit{ParamTail} \mid \varepsilon \\
\mathit{Param} &\to \mathit{Type}\ \texttt{ID} \\
\mathit{Block} &\to \texttt{\{}\ \mathit{StmtList}\ \texttt{\}} \\
\mathit{StmtList} &\to \mathit{Stmt}\ \mathit{StmtList} \mid \varepsilon \\
\mathit{Stmt} &\to \mathit{VarDecl} \mid \mathit{IdStmt} \mid \mathit{IfStmt} \mid \mathit{WhileStmt} \\
&\quad \mid \mathit{InStmt} \mid \mathit{OutStmt} \mid \mathit{ReturnStmt} \mid \mathit{Block} \\
\mathit{VarDecl} &\to \mathit{Type}\ \texttt{ID}\ \mathit{InitOpt}\ \mathit{IdTail}\ \texttt{;} \\
\mathit{IdTail} &\to \texttt{,}\ \texttt{ID}\ \mathit{InitOpt}\ \mathit{IdTail} \mid \varepsilon \\
\mathit{InitOpt} &\to \texttt{=}\ \mathit{Expr} \mid \varepsilon \\
\mathit{IdStmt} &\to \texttt{ID}\ \mathit{IdStmtRest}\ \texttt{;} \\
\mathit{IdStmtRest} &\to \texttt{=}\ \mathit{Expr} \mid \texttt{(}\ \mathit{ArgList}\ \texttt{)} \\
\mathit{IfStmt} &\to \texttt{if}\ \texttt{(}\ \mathit{Expr}\ \texttt{)}\ \mathit{Stmt}\ \mathit{ElsePart} \\
\mathit{ElsePart} &\to \texttt{else}\ \mathit{Stmt} \mid \varepsilon \\
\mathit{WhileStmt} &\to \texttt{while}\ \texttt{(}\ \mathit{Expr}\ \texttt{)}\ \mathit{Stmt} \\
\mathit{InStmt} &\to \texttt{in}\ \texttt{ID}\ \texttt{;} \\
\mathit{OutStmt} &\to \texttt{out}\ \mathit{Expr}\ \texttt{;} \\
\mathit{ReturnStmt} &\to \texttt{return}\ \mathit{ReturnVal}\ \texttt{;} \\
\mathit{ReturnVal} &\to \mathit{Expr} \mid \varepsilon \\
\mathit{Expr} &\to \mathit{Arith}\ \mathit{RelTail} \\
\mathit{RelTail} &\to \mathit{relop}\ \mathit{Arith} \mid \varepsilon \\
\mathit{Arith} &\to \mathit{Term}\ \mathit{ArithTail} \\
\mathit{ArithTail} &\to \mathit{addop}\ \mathit{Term}\ \mathit{ArithTail} \mid \varepsilon \\
\mathit{Term} &\to \mathit{Factor}\ \mathit{TermTail} \\
\mathit{TermTail} &\to \mathit{mulop}\ \mathit{Factor}\ \mathit{TermTail} \mid \varepsilon \\
\mathit{Factor} &\to \texttt{ID}\ \mathit{FactorRest} \mid \texttt{integer} \mid \texttt{char} \\
&\quad \mid \texttt{(}\ \mathit{Expr}\ \texttt{)} \mid \texttt{-}\ \mathit{Factor} \\
\mathit{FactorRest} &\to \texttt{(}\ \mathit{ArgList}\ \texttt{)} \mid \varepsilon \\
\mathit{ArgList} &\to \mathit{Expr}\ \mathit{ArgTail} \mid \varepsilon \\
\mathit{ArgTail} &\to \texttt{,}\ \mathit{Expr}\ \mathit{ArgTail} \mid \varepsilon
\end{aligned}
$$

3.2 Making the grammar LL(1)

The grammar above is already LL(1). Two design choices are responsible:

- **No left recursion.** Repetition is written with right-recursive *tail* nonterminals (*FuncList*, *StmtList*, *ArithTail*, ...), which a recursive-descent parser turns into a simple `while` loop.

- **Disjoint alternatives.** The alternatives of every rule have disjoint FIRST sets, so the next token alone selects the production. The only shared prefix, an identifier at the start of *IdStmt* and *Factor*, is resolved a token later in *IdStmtRest* and *FactorRest*.

FIRST and FOLLOW (abbreviated):

Nonterminal	FIRST	FOLLOW
Program	`int`, `char`	`$`
FuncList	`int`, `char`, ε	`$`
Func	`int`, `char`	`int`, `char`, `$`
Type	`int`, `char`	`ID`
ParamList	`int`, `char`, ε	`)`
Param	`int`, `char`	`,`, `)`
Block	`{`	*FIRST*(*Stmt*), `}`
Stmt	`int`, `char`, `ID`, `if`, `while`, `in`, `out`, `return`, `{`	*FIRST*(*Stmt*), `}`
InitOpt	`=`, ε	`,`, `;`
Expr	`ID`, `integer`, `char`, `(`, `-`	`)`, `;`, `,`
ArithTail	*addop*, ε	*relop*, `)`, `;`, `,`
ArgList	`ID`, `integer`, `char`, `(`, `-`, ε	`)`
ArgTail	`,`, ε	`)`

An excerpt of the resulting LL(1) parsing table; a blank cell is an error entry:

	`int`/`char`	`ID`	`if`	`while`	`{`	`$`
Program	*FuncList*					
FuncList	*Func*					ε
Func	*Type* `ID ( ` *ParamList* ` ) ` *Block*					
Stmt	*VarDecl*	*IdStmt*	*IfStmt*	*WhileStmt*	*Block*	

3.3 Recursive descent

Each nonterminal becomes one method of the `Parser` class, and each right-recursive tail becomes a loop. Because the alternatives are disjoint, the methods read like the grammar itself. The statement dispatcher is a good example:

```
void stmt() {
  if (atType())            varDecl();
  else if (tok.type == ID) idStmt();
  else if (atKeyword("if"))    ifStmt();
  else if (atKeyword("while")) whileStmt();
  else if (atKeyword("in"))    inStmt();
  else if (atKeyword("out"))   outStmt();
  else if (atKeyword("return")) returnStmt();
  else if (atDelim('{'))   block();
  else                     fail("statement");
}
```

The parser keeps a single lookahead token `tok`; it can also print the parse tree as it descends, which is how the tests are checked. On failure it throws a `SyntaxError` carrying the line and column of the offending token, so the driver can report a diagnostic and exit with a non-zero status. Two details deserve a mention:

- the *dangling else* is handled the natural LL(1) way — *ElsePart* is entered only when the next token is `else`, so an `else` always binds to the nearest `if` (as in C and C++);

- a unary minus is folded into *Factor*, which is why `-x` and `-(y * 2)` parse without a separate rule.

4 How the codebase evolved

The work was committed as it happened, so the history is the story of the project. The lexer was built first (September), then the parser was added on top of it (October); the two milestones are tagged `lexer` and `parser`.

The code is split into small modules. Token types and the `Lexer` live behind `include/token.h` and `include/lexer.h` (defined in `src/token.cpp` and `src/lexer.cpp`); the parser lives behind `include/parser.h` (defined in `src/parser.cpp`); and `src/main.cpp` is the command-line driver. The `Makefile` builds the parser and runs the tests, which live under `tests/`.

Date	What changed	Representative commit
8 Sep 2026	project scaffold; operators and delimiters	`chore: initialize ...`, `feat(lexer): ...`
9 Sep 2026	keywords, character literals, relational operators	`feat(lexer): classify keywords ...`
10 Sep 2026	lexical error reporting; sample input; write-up v1; `lexer`	`feat(lexer): report lexical errors ...`
2 Oct 2026	recursive-descent skeleton (functions, parameters, blocks)	`feat(parser): add ...skeleton`
3 Oct 2026	expressions, store statements/calls, declarations	`feat(parser): parse expressions ...`
4 Oct 2026	`if`/`else`, `while`, `in`/`out`/`return`	`feat(parser): parse in, out and return ...`
5 Oct 2026	full test program; write-up v2; `parser`	`test: add a C- program ...`

Through the whole second phase the lexer was never rewritten: the parser simply consumes the token stream it produces. That is exactly the layering the two stages are meant to demonstrate —

$$\text{source text} \xrightarrow{\text{lexer}} \text{tokens} \xrightarrow{\text{recursive-descent parser}} \text{parse tree / diagnostics.}$$